

Postmortem: api date search slow after a migration dropped an index

Incident date	2024-07-09 (Tuesday, afternoon JST)
Severity	SEV-2
Service	api
Incident commander	Mei
Author	Ravi
Status	Final, reviewed 2024-07-11

Summary

api deploy d-4471 carried migration 0193, which dropped the old index on bookings (business_id, starts_at) before its replacement had finished building. Every date search on a booking page then read the bookings table without an index, the primary database ran hot, and date search was slow for 69 minutes. Rolling back the code did not help, because a rollback does not bring back a dropped index. Date search recovered when the new index finished building at 15:19.

Impact

- Customers: date search on booking pages took 2 to 6 seconds between 14:12 and 15:21 JST, and some searches timed out. Support received 212 tickets.
- Bookings made in that window were about 35 % lower than in the same hour the week before.
- Money and data: no payments affected, nothing lost or exposed.

Timeline (JST)

Time	What happened	Who
14:05	api deploy d-4471 goes out. It carries migration 0193, which drops the old index on bookings (business_id, starts_at) and starts building a new composite index in the background.	Ravi
14:09	The background index build starts on the primary database in ap-northeast-1. The old index is already gone, so date searches now read the whole bookings table.	deploy tool
14:12	api p95 latency for the availability endpoint rises from 180 ms to 1.9 s. CPU on the primary database reaches 92 %.	metrics tool
14:16	Alert fires: api p95 above 1.5 s for 3 minutes. The paging tool pages the on-call engineer.	paging tool
14:18	Page acknowledged by Tom, the primary on-call engineer that week.	Tom
14:23	Tom notes the last api deploy is 18 minutes old, and the slow queries in the api logs all read bookings by business and start time.	Tom

Time	What happened	Who
14:27	Incident declared (SEV-2). Mei takes incident commander. Support reports customers saying the booking page spins when they pick a date.	Mei
14:31	Tom rolls api back to the previous deploy, d-4466. The code goes back, but the dropped index does not come back with it.	Tom
14:38	Latency has not recovered seven minutes after the rollback. Tom pages Ravi, because the database now needs the owner's decision.	Tom
14:44	Ravi joins and finds the background index build at 41 %, with about 35 minutes left at its current speed.	Ravi
14:52	Ravi decides not to recreate the old index by hand, because two index builds would compete for the same disk. The team waits for the build.	Ravi
15:06	Support has 190 tickets so far. Mei posts a notice on status.sorrel.app that date search is slow and that bookings still go through.	Mei
15:19	The new index finishes building. p95 for the availability endpoint falls to 240 ms within two minutes.	Ravi
15:24	Ravi deploys d-4471 again, so the code matches the new index.	Ravi
15:40	Mei closes the incident after 15 minutes of normal latency and updates the status page.	Mei

What went well

- Tom connected the slow queries to the 18-minute-old deploy within seven minutes of the page.
- Ravi's call not to build a second index by hand kept the disk from becoming the next problem.

What went badly

- The migration dropped the old index before the new one existed. Rolling back the code could not undo that, and nobody on the incident knew it until the rollback had failed.
- The api-wide p95 alert hid how slow the availability endpoint alone had become.

Action items

Action	Owner	Due	Status
Never drop an index in the same deploy that builds its replacement: drop the old one in a later deploy, after the new one is ready.	Ravi	2024-07-19	Done
Run index builds on tables over 10 million rows outside 09:00 to 21:00 JST.	Ravi	2024-07-26	Done
Give the availability endpoint its own p95 alert, separate from the api-wide one.	Tom	2024-08-02	Open
Add a line to the api runbook: a rollback does not undo a migration.	Ravi	2024-07-31	Done